Introduction

This project implements a scalable, cloud-based image annotation application using a hybrid AWS architecture combining EC2 and serverless components. The web application is deployed on Amazon EC2 instances managed by an Auto Scaling Group (ASG) within private subnets of a custom VPC. These instances are accessed via an Application Load Balancer (ALB) residing in public subnets, ensuring high availability and secure access. Uploaded images are stored in an S3 bucket, which triggers two AWS Lambda functions, one for thumbnail generation using Pillow and another for caption generation using the Google Gemini. Metadata, including filenames, thumbnails, captions and upload time, is stored in an Amazon RDS MySQL database. The architecture ensures secure and seamless integration between the EC2-based frontend and the serverless backend through shared access to S3 and RDS, while enforcing fine-grained control through IAM roles, security groups, and a bastion host for administrative access.

Architecture Diagram

Web Application Architecture

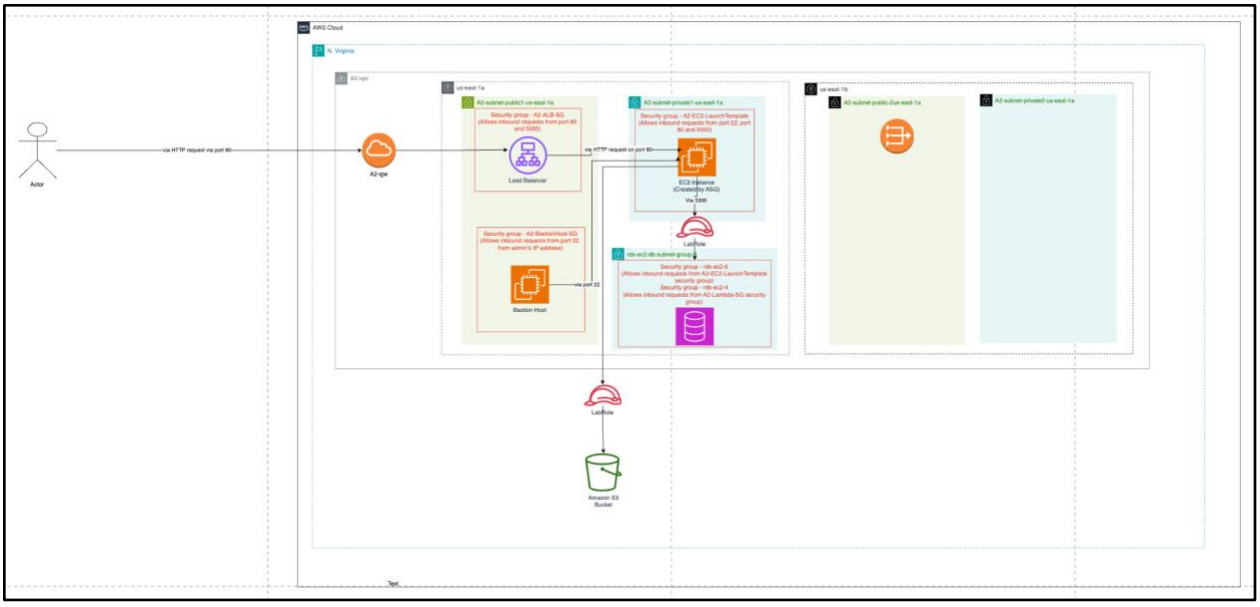


Fig – 1

Serverless Architecture

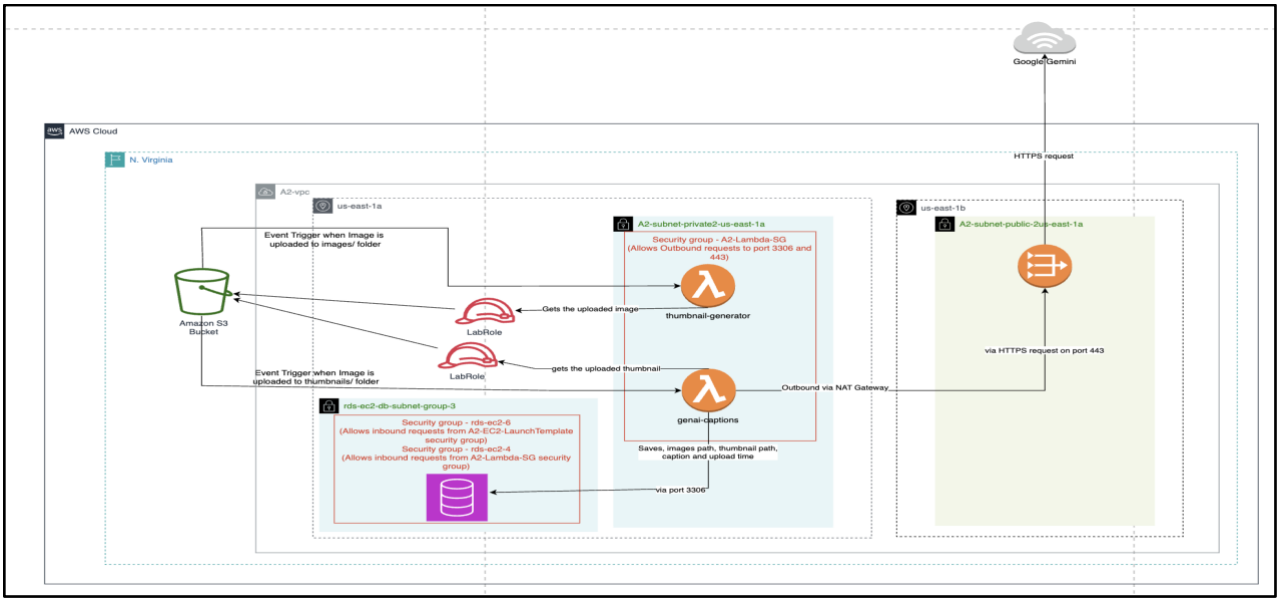


Fig – 2

Web Application Deployment

Compute Environment

The web application is deployed on Amazon EC2 instances managed by an Auto Scaling Group (ASG) to ensure high availability and responsiveness to traffic fluctuations. Initially, an EC2 instance - A2-EC2 was configured with the application code and its required dependencies. A custom Amazon Machine Image (AMI) - A2-EC2-Image was then created from this instance and used in the launch template of the ASG - A2-Ec2-AutoScalingGroup. This setup ensures that any new instances launched by the ASG start with the correct environment and application setup. A user data script within the launch template is used to automatically start the Flask-based web application upon instance launch.

The EC2 instances operate within a custom Virtual Private Cloud (VPC) - A2-vpc, which spans multiple Availability Zones, us-east-1a and us-east-1b, and includes both public and private subnets. In this architecture, the web servers are launched into private subnets, increasing security by preventing direct internet access to the instances. All external traffic is routed through an Application Load Balancer (ALB) - A2-LoadBalancer that resides in a public subnet. The ALB handles incoming HTTP requests from the internet and forwards them to the EC2 instances in the private subnets.

To enable outbound internet access, a NAT Gateway - A2-NATGateway is deployed in a public subnet and associated with the private subnets via route tables. The private subnets' route tables are configured to forward internet-bound traffic through the NAT Gateway, while the public subnets are associated with a route table connected to the Internet Gateway - A2-igw to support direct access for the ALB. This design ensures that the EC2 instances running the web application are not directly exposed to the internet, while still remaining fully functional through the ALB for handling user requests.

Security is enforced using security groups and IAM roles. The EC2 security group permits inbound traffic on port 80 for HTTP, port 5000 for Flask app and port 22 for SSH. The Application Load Balancer (ALB) also has its own security group, allowing HTTP traffic from anywhere - 0.0.0.0/0. The LabRole IAM roles attached to the EC2 instances grant permission to access the RDS database and the S3 bucket.

Administrative access to the EC2 instances is provided via a bastion host located in the public subnet. This host allows SSH access from the developer's IP address and can be used to securely connect to other instances within private subnets, if necessary.

EC2 Auto Scaling Group (ASG) Configuration

- **Auto Scaling Group Name:** A2-AutoScalingGroup
- **AMI used:** A2-EC2-Image
- **Launch Template**: A2-EC2-LaunchTemplate . (Refer to Fig - 3)
    - It Includes user-data script to run Flask app -

      ```
      #!/bin/bash
      cd /home/ec2-user/Image-caption-app-V1.1
      nohup python3 app.py > app.log 2>&1 &
      ```

- **Instance Type**: t2.micro
- **Desired Capacity**: 1 (Refer to Fig - 3)
- **Min Capacity**: 1 (Refer to Fig - 3)
- **Max Capacity**: 3 (Refer to Fig - 3)
- **Scaling Policy**: Scale out at CPU > 60%, scale in at CPU < 60% (Refer to Fig - 4)

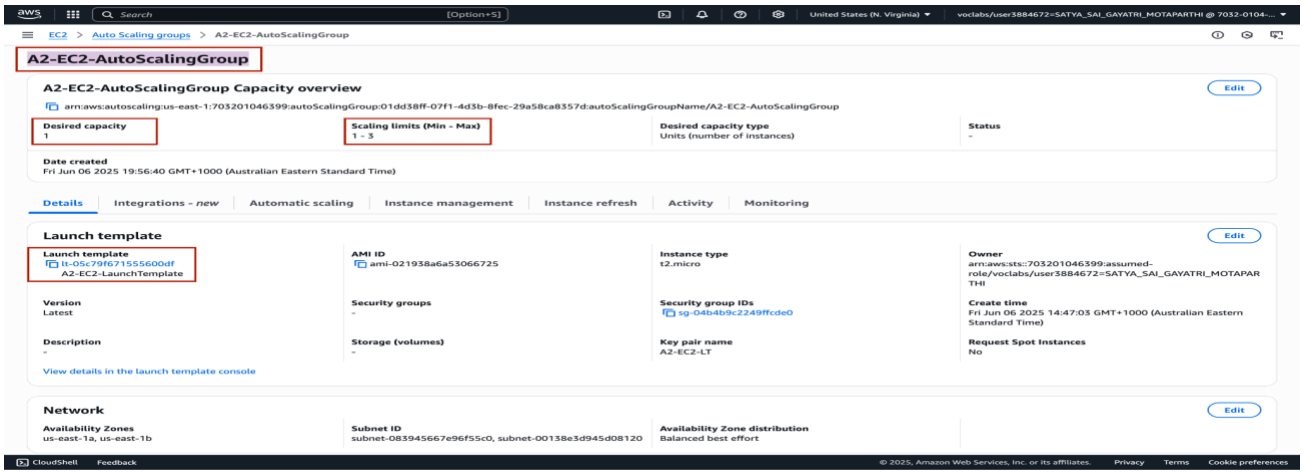


Fig – 3

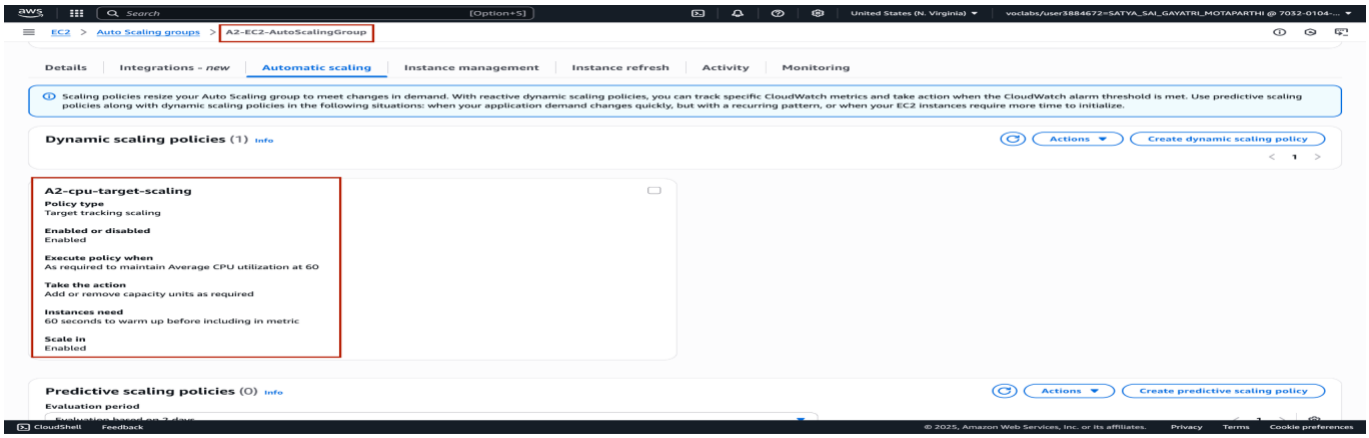


Fig - 4

Network Settings

- **VPC**: A custom VPC - A2-vpc was created with 2public and 2private subnets (Refer to Fig - 5)
- **Subnets**: The EC2 instances launched by the Auto Scaling Group are launched in private subnets of the custom VPC - A2-vpc.
- **Internet Gateway**: The Internet Gateway - A2-igw is attached to custom VPC - A2-vpc for public access (Refer to Fig - 5)

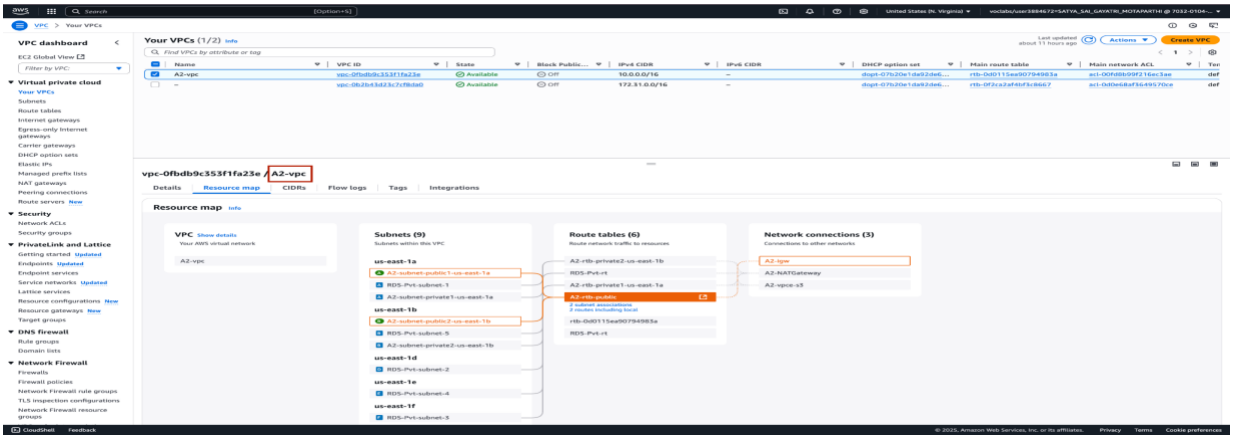


Fig - 5

Security Configurations

- **Security Groups**:
  - EC2 security group - A2-EC2-LaunchTemplate allows HTTP - port 80, Custom TCP - port 5000 and SSH - port 22. (Refer to Fig - 6)
  - ALB security group- A2-ALB-SG allows HTTP - port 80 and Custom TCP - port 5000 from 0.0.0.0/0 (Refer to Fig - 7)

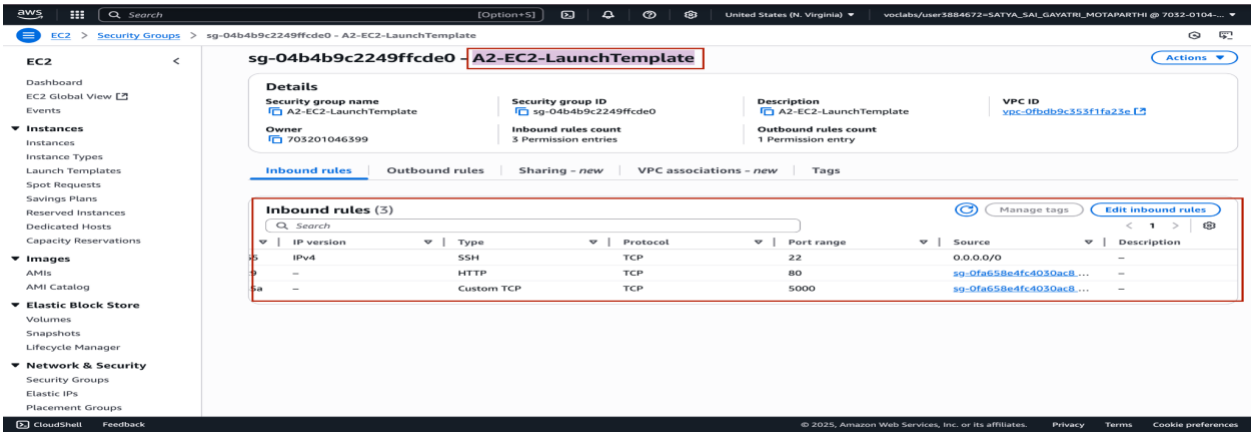


Fig - 6

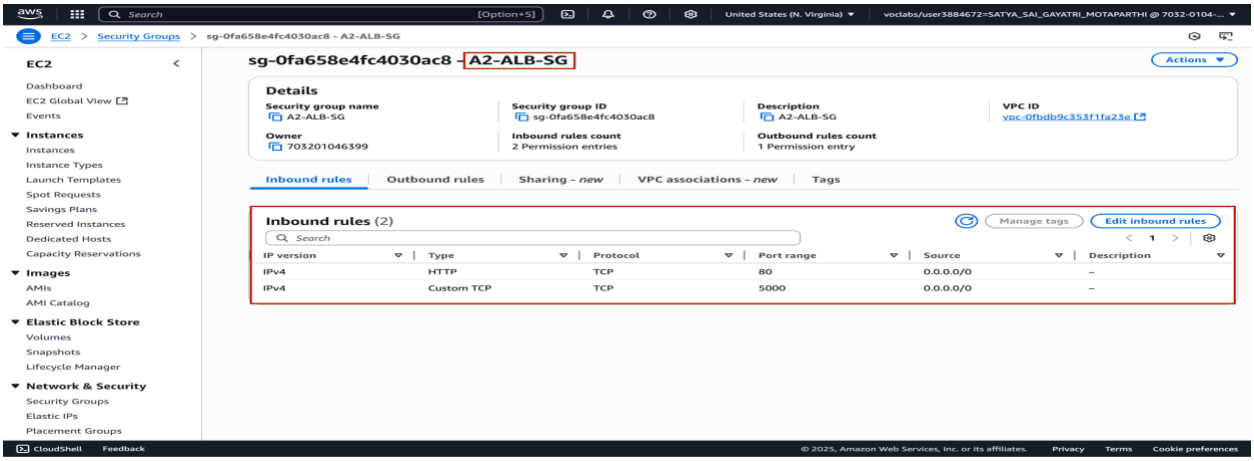


Fig - 7

- **IAM Roles**:
  - The EC2 IAM role - LabRole is attached with policies to allow RDS, S3 access (Refer to Fig - 8)

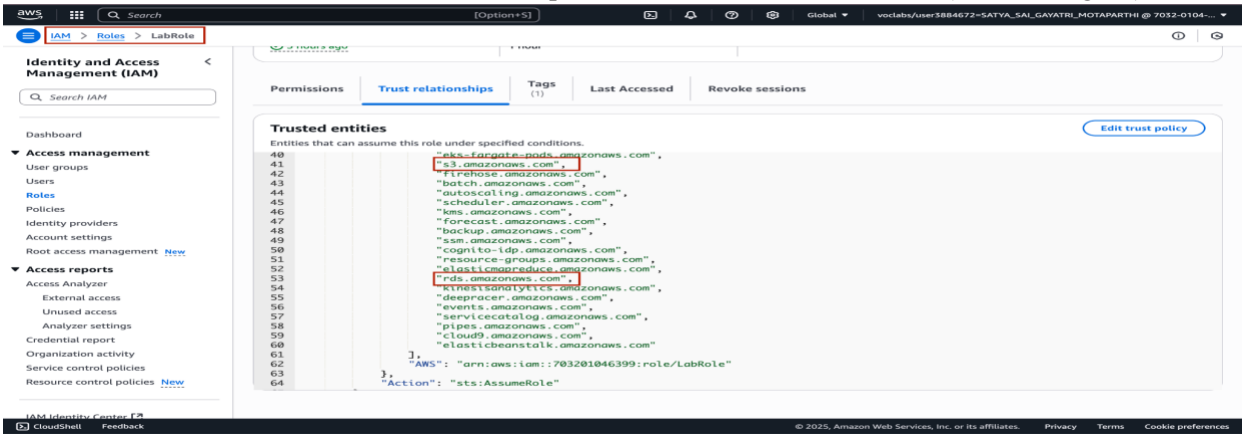


Fig – 8

Administrative Access

- **Bastion Host**:
  - It is launched in the public subnet on the custom VPC - A2-vpc
  - It is used for SSH access to EC2 instances created by the Auto Scaling Group - A2-AutoScalingGroup in the private subnet
  - The Security group - A2-BastionHost-SG restricts SSH access to the IP of the admin (In this case - my personal laptop)

Load Balancer Setup

An Application Load Balancer (ALB) is used to distribute incoming HTTP traffic across all EC2 instances in the Auto Scaling Group. The ALB - A2-LoadBalancer is configured with a listener on port 80, and it forwards requests to a target group - Assignment2-TargetGroup consisting of the EC2 instances. Health checks are configured to monitor the root path (/) of the application, with a success response code of 200. The ALB - A2-LoadBalancer ensures load is evenly distributed and contributes to fault tolerance by automatically routing requests away from unhealthy instances. It also plays a vital role in enabling scalability by directing traffic to new instances as they are launched by the ASG - A2-AutoScalingGroup.

ALB Configuration

- **Listener**: HTTP (port 80) (Refer to Fig - 9)
- **Target Group**: EC2 ASG instances registered (Refer to Fig - 10)
- **Health Checks**: Path: /, Success code: 200 (Refer to Fig - 10)

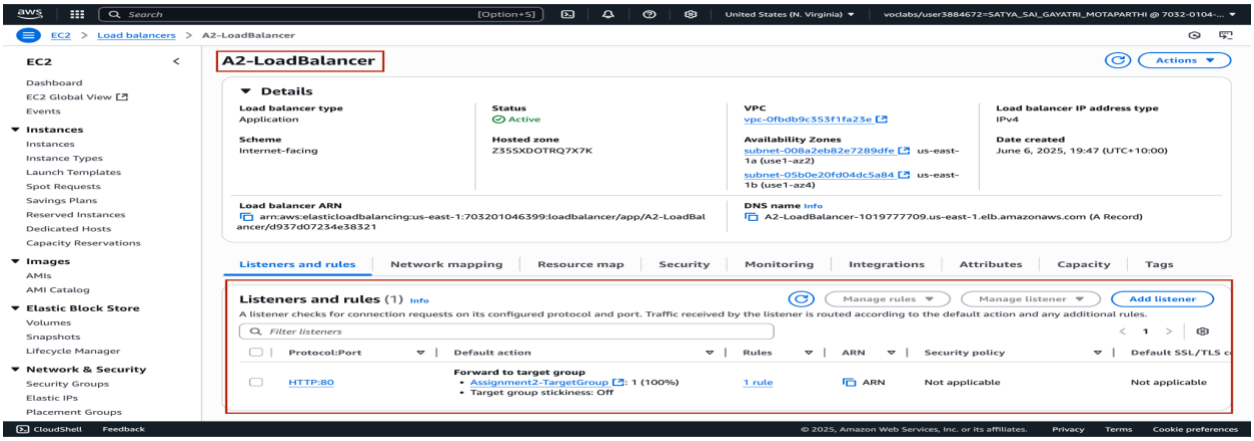


Fig – 9

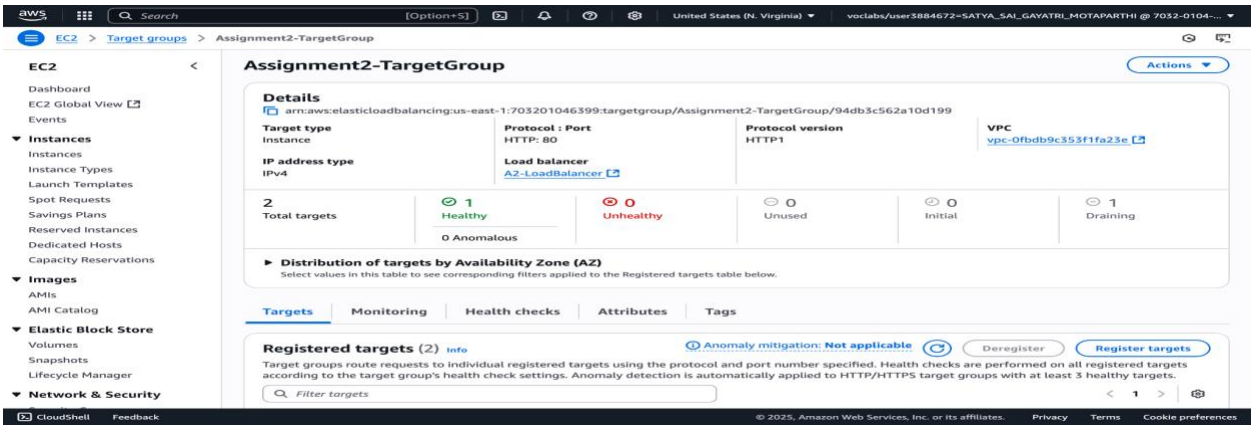


Fig – 10

Database Environment

The metadata for uploaded images, such as filenames, captions, thumbnails and timestamps, is stored in an Amazon RDS MySQL database instance - Assignment2-RDS. The RDS instance is deployed within the custom VPC - A2-vpc, in us-east-1a availability zone, using rds-ec2-db-subnet-group-3 DB subnet group that spans five subnets for availability. The instance is configured with the db.t4g.micro instance type. It is placed in a private subnet to enhance security, and only the EC2 instances and Lambda functions are permitted to access it through the rds-ec2-6 and rds-ec2-4 security groups. Access is restricted by referencing the security group ID of the EC2 instances and Lambda Functions rather than using IP addresses. Application credentials for the RDS database are added to the image-caption-app codbase for the EC2 Instances and in the lambda_handler() function of the Lambda Functions to be accessed during database connections. Below is the configuration of the RDS Database:

Name: Assignment2-RDS

1. **Engine:** MySQL (Refer to Fig - 11)
2. **Instance Class:** db.t4g.micro (Refer to Fig - 11)
3. **VPC:** A2-vpc (Refer to Fig - 11)
4. **Availability Zone:** us-east-1a (Refer to Fig - 11)
5. **Subnet Group:** rds-ec2-db-subnet-group-3 (Refer to Fig - 11)
    a. Subnets: 5 subnets
6. **Private:** Yes (Refer to Fig - 11)
7. **Security Group:** (Refer to Fig - 11)
    a. rds-ec2-6: lets the EC2 instances in the ec2-rds-6 security group access it via port 3306 (Refer to Fig - 15)
    b. rds-ec2-4: lets the Lambda functions in the A2-Lambda-SG security group access it via port 3306 (Refer to Fig - 13)
8. **Integration:** The Flask app, using MySQL-Connector, and the Lambda functions, using pymysql, connect to the RDS via the credentials added in the codebase.

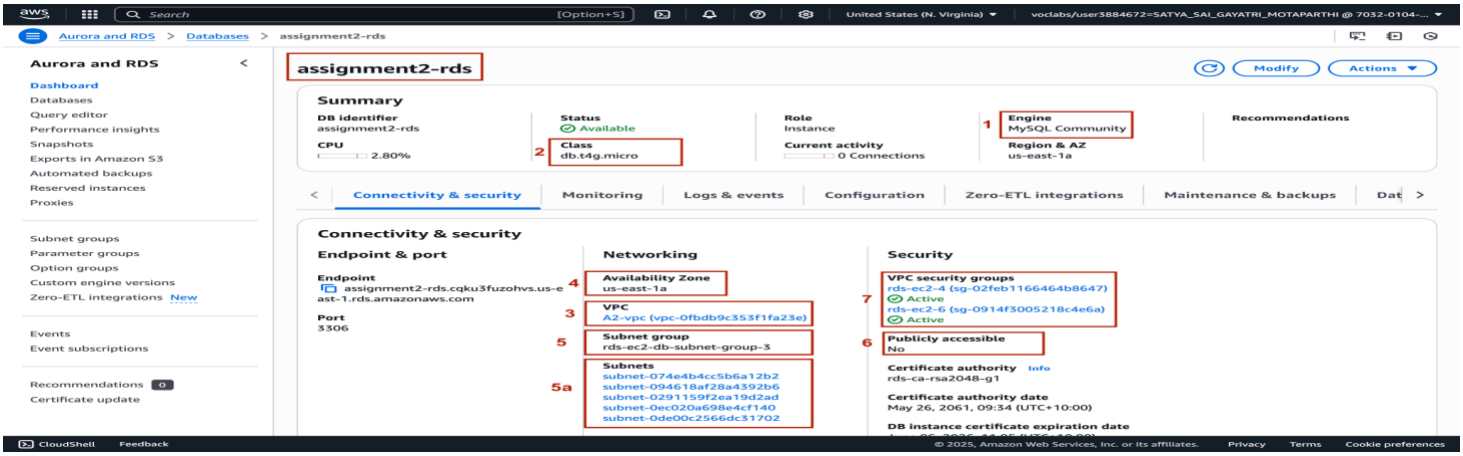


Fig – 11

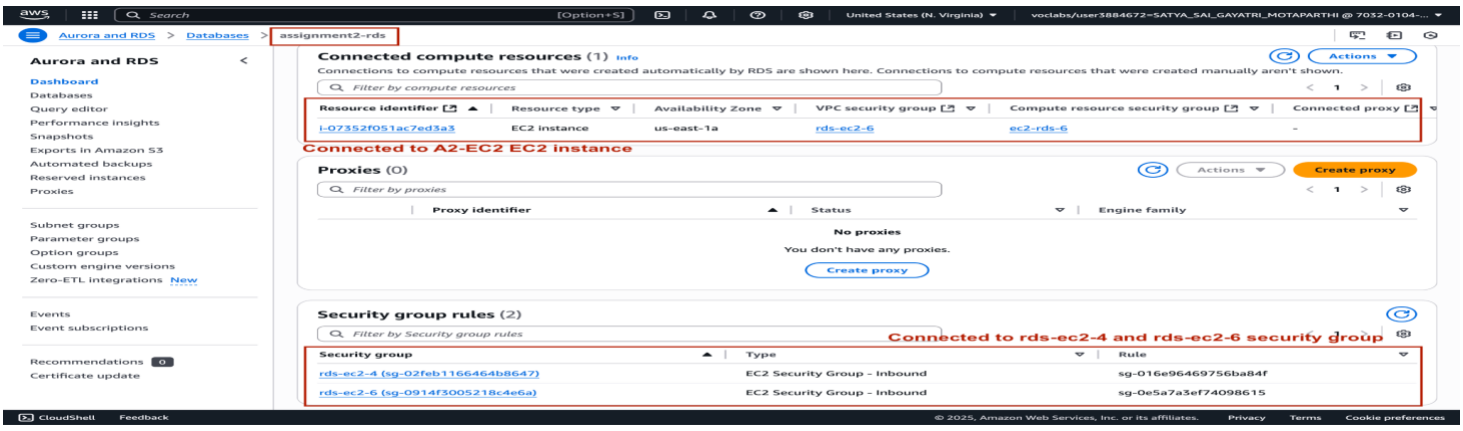


Fig – 12

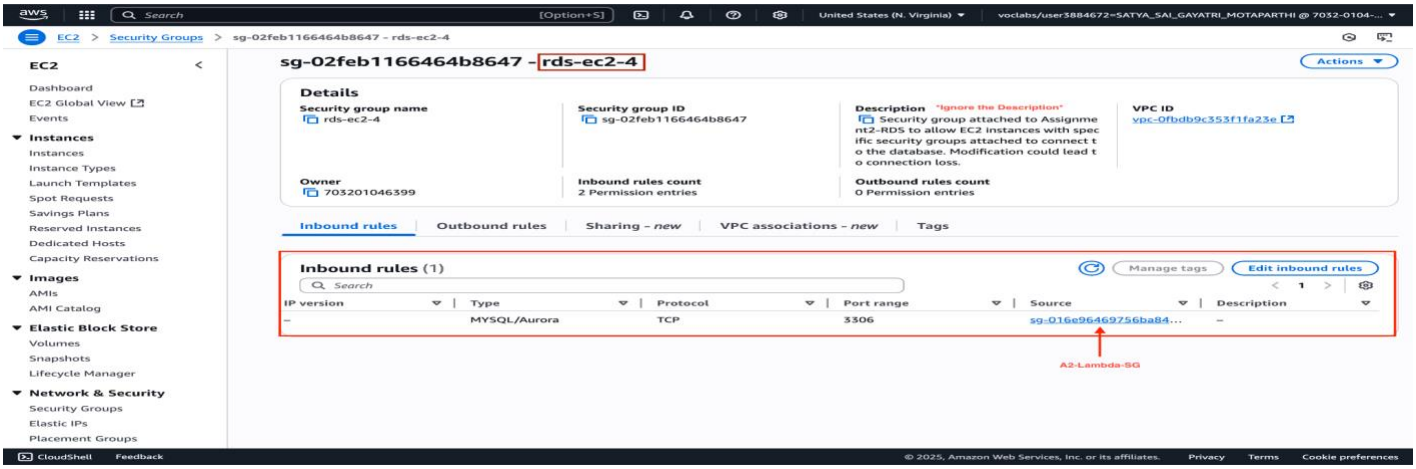


Fig – 13

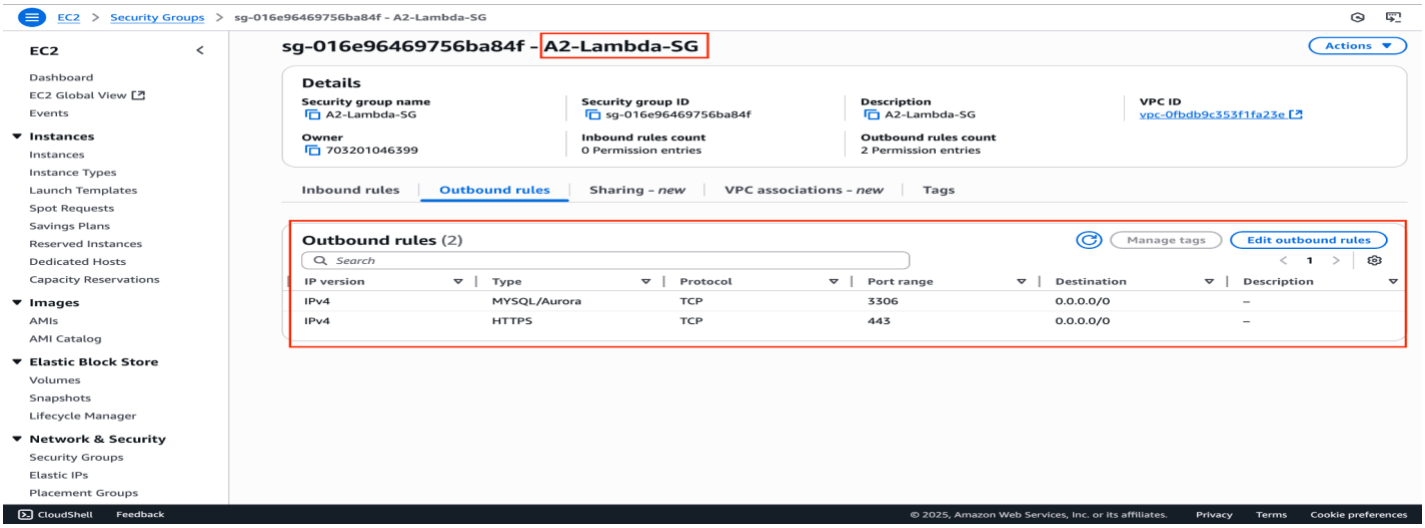


Fig - 14

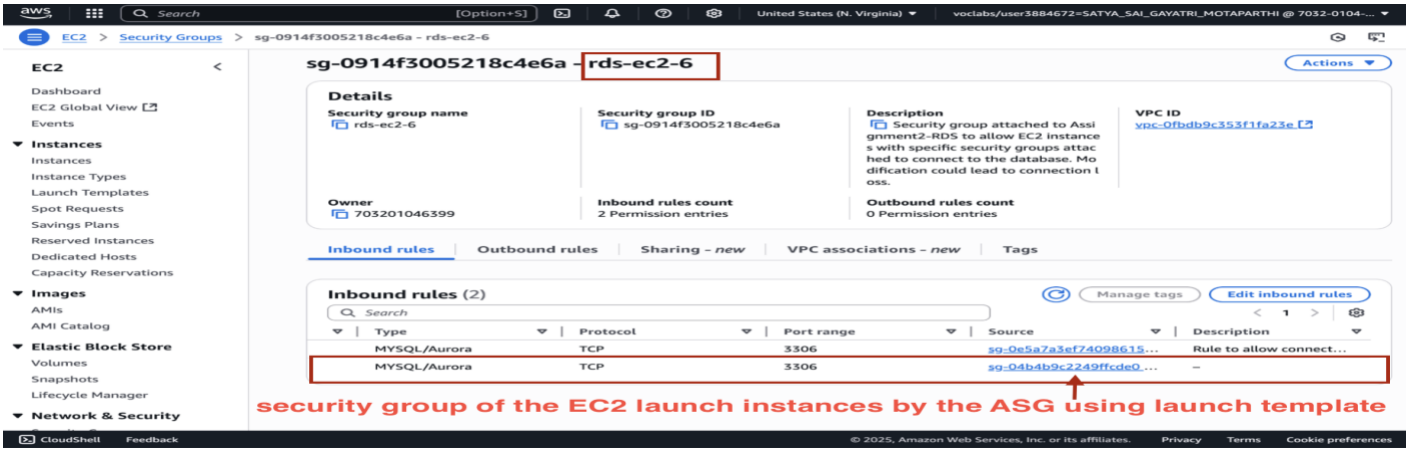


Fig – 15

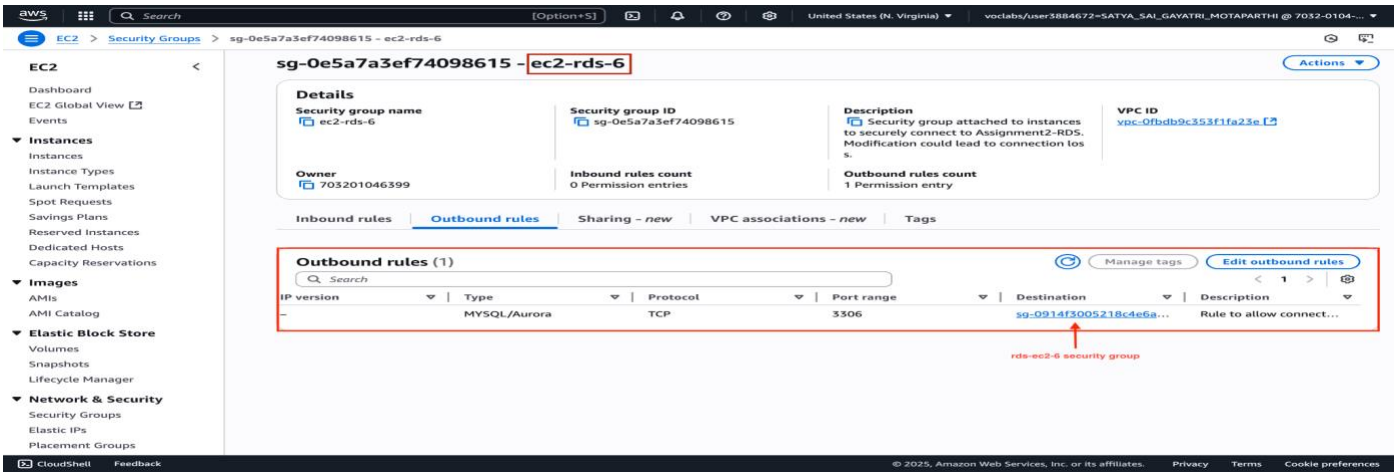


Fig – 16

Storage Environment

Uploaded images are stored in an Amazon S3 bucket - a2-s3-webapp, organised under an images/ prefix. This S3 bucket serves as both a storage backend and a trigger source for the serverless components. When an image is uploaded, S3 generates a PUT event, which automatically triggers the caption/annotation and thumbnail generation Lambda functions. The thumbnail images generated by the thumbnail-generator Lambda function are saved in the same bucket under a thumbnails/ prefix. Proper IAM permissions are set by assigning the LabRole IAM role to both Lambda functions and EC2 instances to access S3. The bucket's event notification configuration is used to bind the upload action with the respective Lambda triggers, ensuring real-time processing as new files arrive.

1. **a2-s3-webapp S3 Bucket**:
    a. Stores uploaded images under images/ (Refer to Fig - 17)
    b. Triggers Lambda functions on PUT events (Refer to Fig - 18)
2. **Thumbnails Folder**:
    a. Lambda function saves thumbnails in thumbnails/ prefix (Refer to Fig - 17)
3. **Bucket Policies:**
    a. Allows access from Lambda and EC2 via LabRole IAM Role (Refer to Fig - 19)

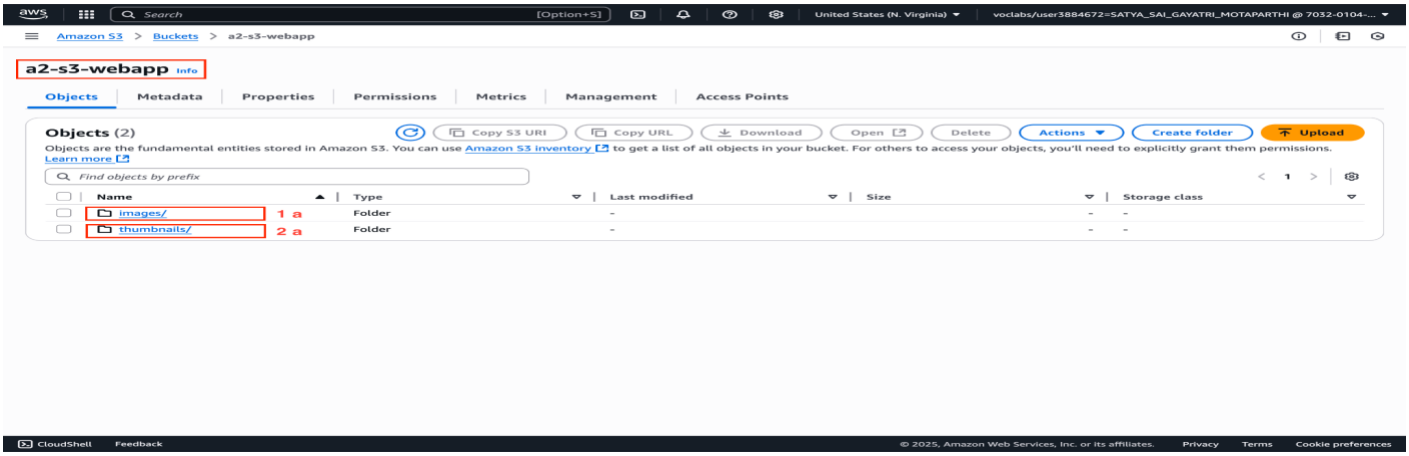


Fig – 17

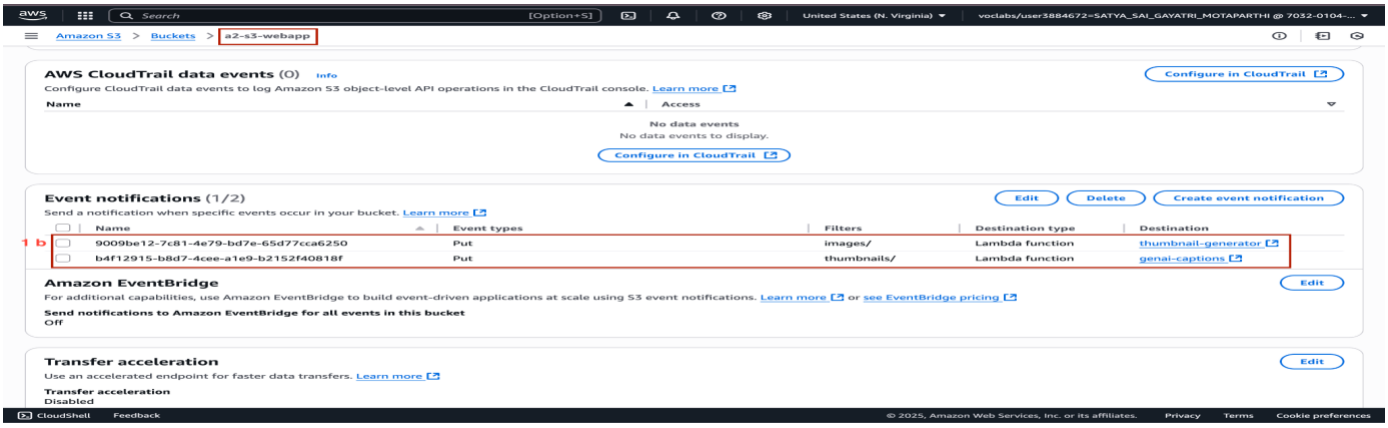


Fig – 18

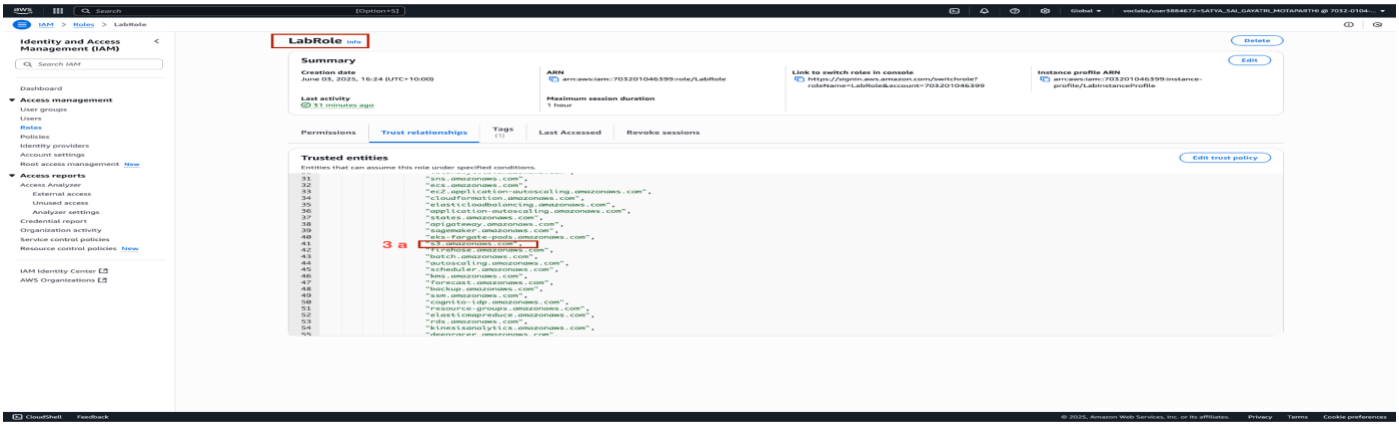


Fig – 19

Serverless Component Deployment

Event-Driven Architecture

This section describes how the serverless Lambda functions are automatically triggered in response to specific S3 events and how they interact with other AWS resources.

- **Trigger Mechanism**:
  The Lambda functions are triggered using Amazon S3 PUT events. Whenever a new image is uploaded to the specified S3 bucket, these events invoke the relevant functions. (Refer to Fig - 18)
- **S3 Event Configuration**:
  The S3 bucket has an event notification rule configured to trigger:
  - The Thumbnail Lambda Function upon image upload - images/ prefix (Refer to Fig - 18)
  - The Caption/Annotation Lambda Function upon thumbnail upload - thumbnails/ prefix (Refer to Fig - 18)
- **Resources Accessed by the Functions**:
  - **Thumbnail Generator Lambda**:
    - Reads image from S3 from images/
    - Generates a thumbnail using Pillow
    - Saves thumbnail to thumbnails/ folder in the same S3 bucket
  - **Caption/Annotation Lambda**:
    - Reads image from S3 from thumbnails/
    - Generates a caption using Google Gemini
    - Writes metadata like image path, thumbnail path, caption and the upload time into RDS.

Thumbnail Generator

This Lambda function is responsible for generating image thumbnails when a new image is uploaded to the S3 bucket.

- **Function Name**: thumbnail-generator
  Packaging Method:
  - This function is created directly via the AWS Management Console
  - The function code is written in Python
  - An external dependency - Pillow has been added via a Lambda layer (Refer to Fig - 21)
    - Layer Name: pillow-layer
    - Layer ARN: arn:aws:lambda:us-east-1:703201046399:layer:pillow-layer:1
    - It is deployed to the same region as the Lambda using the AWS Serverless Application Repository
- **Deployment Method**:
  - The function code was written directly in the inline code editor in the Lambda Console.
  - The Pillow layer was added via the Layers section in the Lambda Console.
- **Trigger Source**: (Refer to Fig - 23)
  - The S3 bucket event on PUT for images in images/ folder triggers this function.
- **VPC Settings**: (Refer to Fig - 22)
  - VPC: A2-vpc, the custom VPC created.
  - Subnets: us-east-1b: A2-subnet-private2-us-east-1b and us-east-1a: A2-subnet-private1-us-east-1a
  - Security Group: A2-Lambda-SG, It allows outbound MySQL - port 3306 traffic to the RDS instance and HTTPS - port 443 in case it requires to connect to the Google Gemini.
- **Function Logic Summary**:
  - This function is triggered when a new image is uploaded to images/ folder in the a2-s3-webapp S3 bucket.
  - Then downloads the image from the S3 bucket using boto3.
  - It uses Pillow to create a thumbnail of the image.
  - It then uploads the thumbnail to thumbnails/ folder in the a2-s3-webapp S3 bucket.
    Permissions:
  - The LabRole IAM Role's trust policy shows that the user has permission to access the S3 bucket.
- **Environment Variables**
  - The environment variables are configured via the Console.
  - S3_BUCKET: a2-s3-webapp

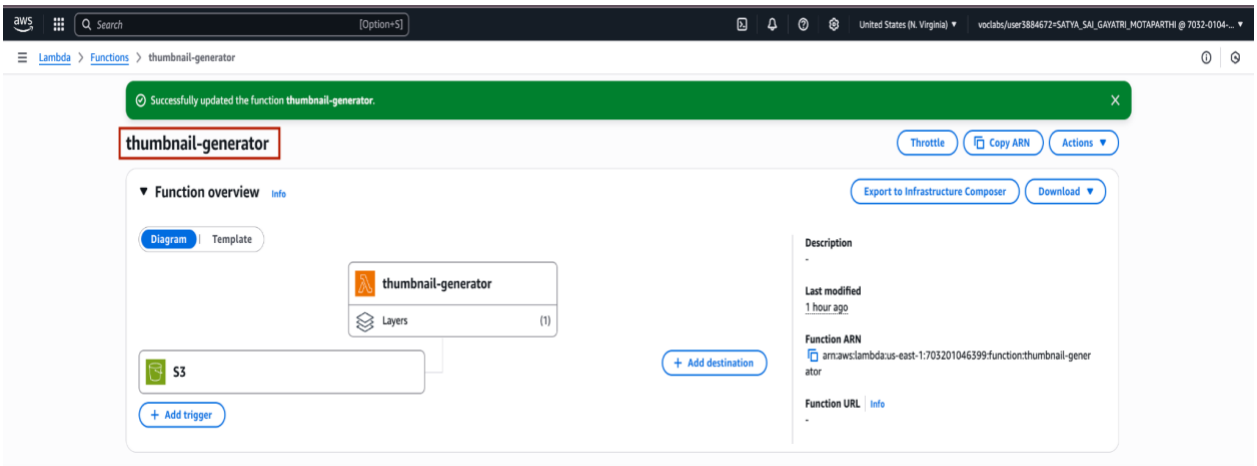


Fig - 20

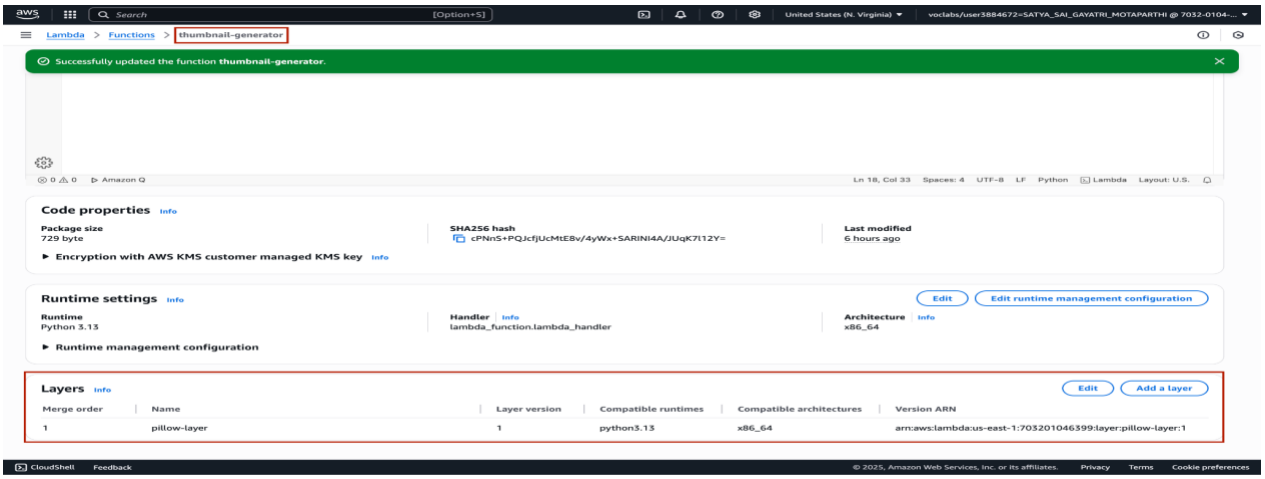


Fig – 21

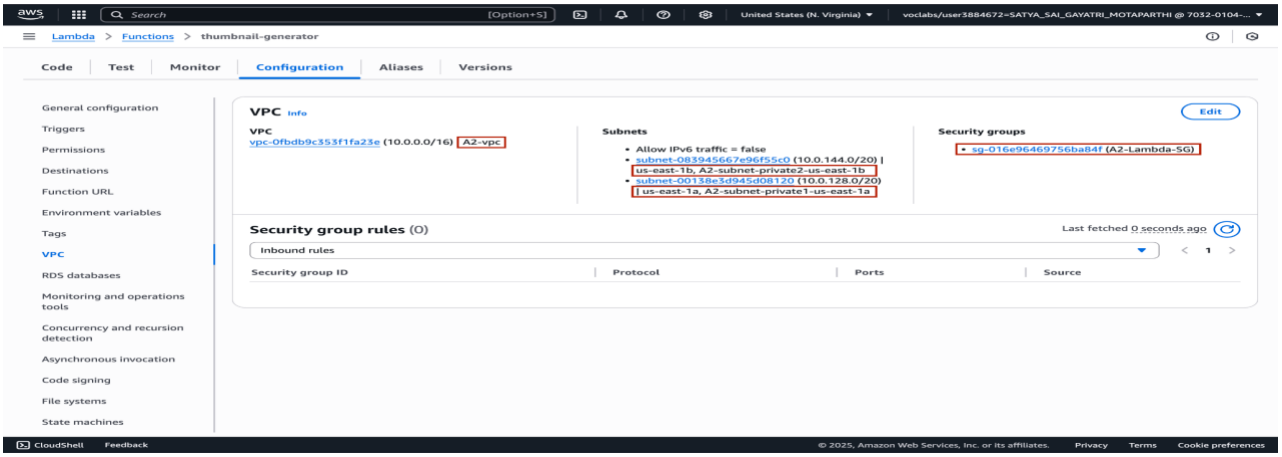


Fig – 22

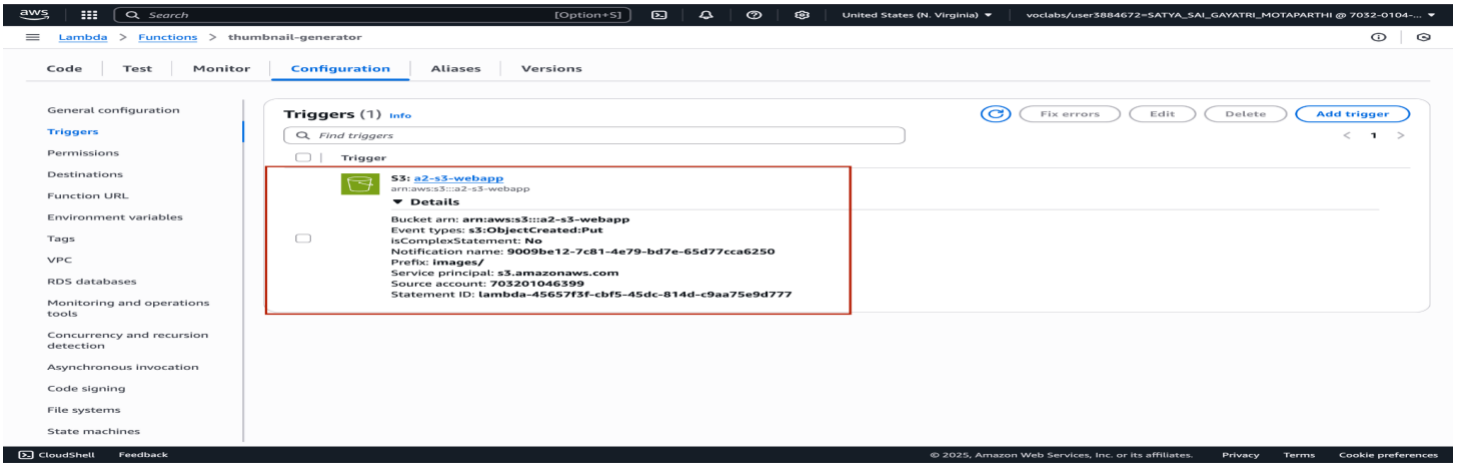


Fig - 23

Caption/Annotation Function

This Lambda function is responsible for generating image captions using the Google Gemini and saving the results in an RDS MySQL database.

- **Function Name**: genai-captions
- **Packaging Method**: Docker container image (Refer to Fig - 24)

- It is built locally with a custom Dockerfile
- It includes all the necessary dependencies: google-generativeai and pymysql.
- The image is pushed to the Amazon ECR repository.

- **Deployment Method**: **AWS CLI** (Refer to Fig - 24)
  - Created using the following commands:
    aws lambda create-function \
      --function-name genai-captions \
      --package-type Image \
      --code ImageUri=$ACCOUNT_ID.dkr.ecr.us-east-1.amazonaws.com/genai-captions-generator:latest \
      --role arn:aws:iam::accountId:role/LabRole \
      --region us-east-1
- **Trigger Source**: (Refer to Fig - 26)
  - The S3 bucket event on PUT for images in thumbnails/ folder triggers this function.
- **VPC Settings**: (Refer to Fig - 25)
  - VPC: A2-vpc, the custom VPC created.
  - It is required for the Lambda function to be in the same VPC as the RDS, as the RDS is present in the private subnet of the VPC.
  - Subnets: us-east-1b: A2-subnet-private2-us-east-1b and us-east-1a: A2-subnet-private1-us-east-1a
  - Security Group: A2-Lambda-SG, It allows outbound MySQL - port 3306 traffic to the RDS instance and HTTPS - port 443 to connect to the Google Gemini.
- **Function Logic Summary**:
  - This function is triggered when an image is uploaded to the thumbnails/ in a2-s3-webapp S3 bucket.
  - Then downloads the image from the S3 bucket using boto3.
  - It uses the google-generativeai to generate a caption.
  - It then inserts the image filename, thumbnail filename and caption into the RDS MySQL database.
- **Permissions**:
  - The LabRole IAM Role's trust policy shows that the user has permission to access the S3 bucket.
- **Environment Variables**:
  - The environment variables are configured via the Console.
  - GEMINI_API_KEY
  - DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME

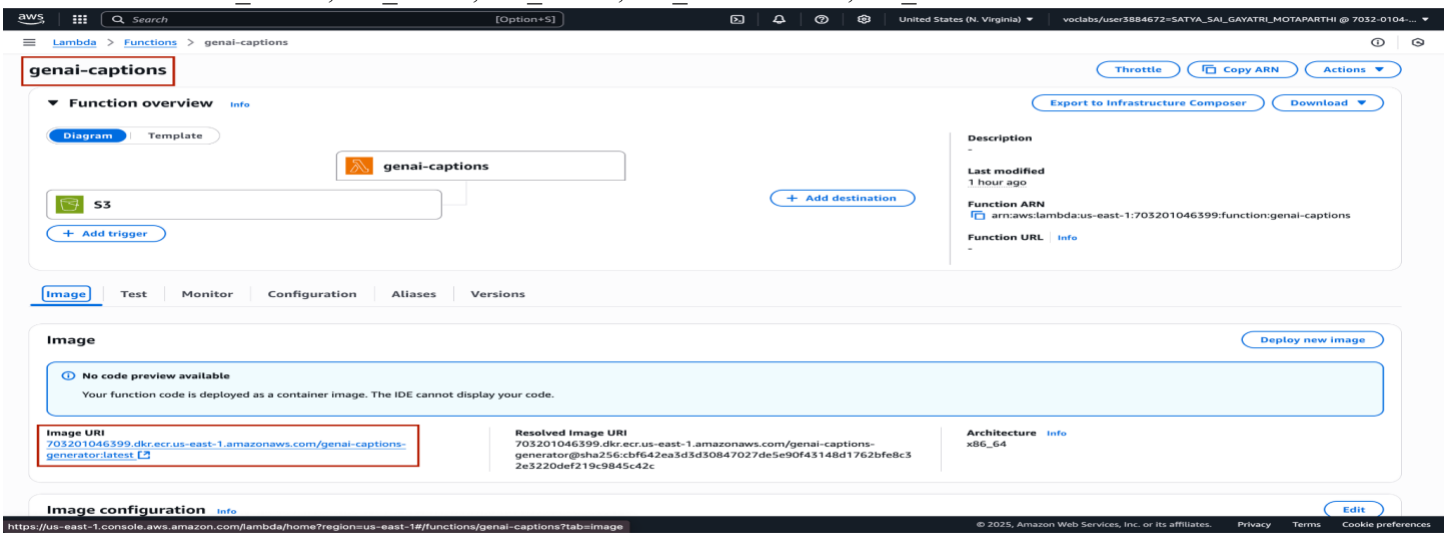


Fig – 24

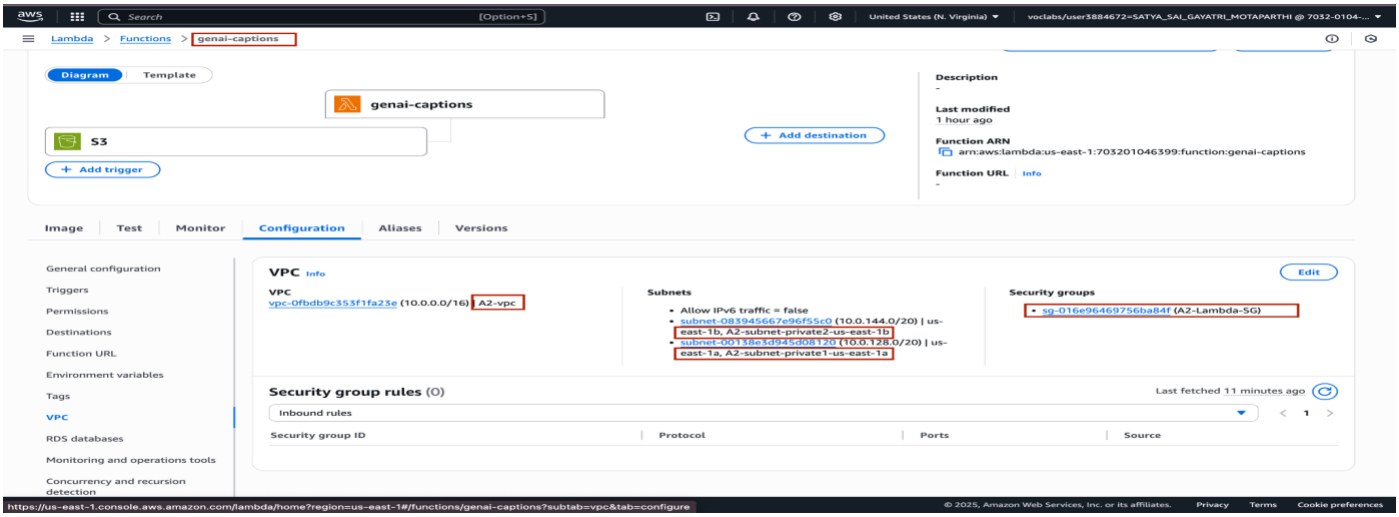


Fig - 25

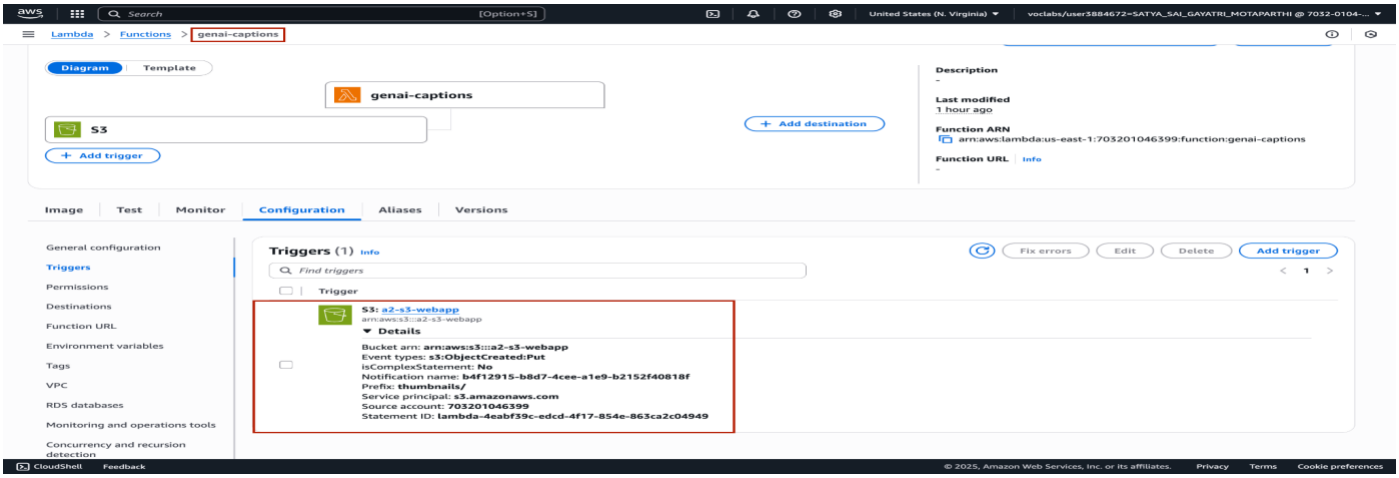


Fig – 26

Summary and Lessons Learned Conclude

This assignment successfully demonstrates the deployment of a secure and scalable image annotation system using AWS services. The Auto Scaling Group and ALB provided a robust and elastic compute environment, while S3-triggered Lambda functions enabled real-time processing of uploaded content. A key learning experience involved configuring VPC access for Lambda functions to securely connect with the RDS database, which required careful setup of subnets, route tables, and security groups. Additional challenges included packaging Lambda functions with external dependencies, which were resolved using Docker containers and Lambda layers. Overall, the project offered valuable hands-on experience in integrating compute and serverless resources within a cloud-native architecture.